

Projekt Zaliczeniowy - PSAM

Moje Miasteczko (My Little Town)

Cyfrowa implementacja gry planszowej "Rolling Village"

Autorzy: Marek Otulak, Sebastian Kałuża **Data:** 19 listopada 2025

Przedmiot: PSAM

1. Wstęp

1.1 Opis projektu

"Moje Miasteczko" to cyfrowa wersja gry planszowej "Rolling Village". Gra polega na budowaniu miasteczka przez 9 rund, w których gracz rzuca kośćmi i umieszcza różne typy projektów (domy, lasy, stawy, place, fabryki) na planszy 5×6. Punkty zdobywa się przez tworzenie grup połączonych projektów tego samego typu oraz strategiczne budowanie placów i fabryk.

Głównym wyzwaniem było przeniesienie złożonych zasad gry planszowej do interaktywnej aplikacji webowej, zachowując przy tym płynność rozgrywki i intuicyjny interfejs.

1.2 Setup środowiska

Wymagania:

- Node.js w wersji 20.19.0 lub 22.12.0+
- npm (Node Package Manager)

Instalacja projektu:

```
# 1. Sklonuj repozytorium
git clone https://github.com/TurboSebo/my-little-town.git
cd my-little-town

# 2. Zainstaluj zależności
npm install

# 3. Uruchom serwer deweloperski
npm run dev

# 4. Otwórz w przeglądarce
# Aplikacja dostępna pod adresem: http://localhost:5173
```

Dodatkowe komendy:

```

npm run build      # Build produkcyjny
npm run preview    # Podgląd buildu
npm run type-check # Sprawdzenie typów TypeScript
npm run lint       # Sprawdzenie kodu (ESLint)
npm run format     # Formatowanie kodu (Prettier)

```

1.3 Stack technologiczny

- **Vue 3** (v3.5.22) - framework frontendowy z Composition API
- **TypeScript** (v5.9.0) - typowanie statyczne
- **Pinia** (v3.0.3) - state management (Vuex następnej generacji)
- **Vite** (v7.1.11) - build tool i dev server z HMR
- **Vue Router** (v4.6.3) - routing (przygotowany na przyszłość)
- **ESLint + Prettier** - quality & formatting

Dlaczego ten stack?

Wybraliśmy Vue 3 z TypeScript, bo daje świetne wsparcie typów i łatwą kompozycję logiki. Pinia to prostsza alternatywa dla Vuex, która lepiej współpracuje z TypeScript. Vite to bardzo szybki bundler - rebuild trwa milisekundy, co przyspiesza development.

1.4 Struktura projektu

```

my-little-town/
├── src/
│   ├── components/           # Komponenty Vue
│   │   ├── BoardGrid.vue     # Główna plansza gry (siatka 5x6)
│   │   ├── DiceRoller.vue    # Kości i przyciski akcji
│   │   ├── GameBoard.vue     # Kontener - łączy wszystkie komponenty
│   │   ├── GameInfo.vue      # Info o rundzie i punktach
│   │   ├── GameLegend.vue    # Legenda projektów (wybór typu)
│   │   ├── GameStartModal.vue # Modal startowy
│   │   ├── GameEndModal.vue  # Modal końcowy z wynikami
│   │   └── RoundScores.vue    # Tabela punktów z rund
│   ├── stores/
│   │   └── GameStore.ts       # Logika gry (1393 linii)
│   ├── router/
│   │   └── index.ts           # Routing (placeholder)
│   ├── assets/                # Style CSS
│   │   ├── board.css         # Style planszy i komórek
│   │   ├── ui.css            # Style UI (przyciski, legendy)
│   │   └── main.css          # Layout główny
│   ├── App.vue               # Root component
│   └── main.ts                # Entry point
└── public/                   # Statyczne pliki

```

2. Zasady gry

2.1 Podstawowy przebieg

Gra składa się z fazy planowania, **9 rund** plus faza końcowa. W każdej rundzie:

1. **Rzut kośćmi** - wynik określa, w których kolumnach można stawiać projekty
2. **Wybór projektu** - z legendy po lewej stronie
3. **Umieszczenie** - kliknięcie w odpowiednie pole na planszy
4. **Zapisanie zmian** - potwierdzenie ruchów
5. **Następna runda** - przejście dalej (co 3. runda = bonus)

2.2 Typy projektów

Ikona	Typ	Warunek kostki	Punktacja
	Dom	1 lub 4	Grupowanie
	Las	2 lub 5	Grupowanie
	Staw	3 lub 6	Grupowanie
	Plac	Dubel oczek (np. 3,3)	+10 jeśli sąsiaduje z domem, lasem I stawem
	Fabryka	Dubel projektowy (np. 1,4)	+10 za las/staw, -2/-5 za dom/plac

2.3 Mapowanie kości na wiersze

Suma kości określa, który **wiersz** (ulica) jest punktowany w danej rundzie:

- Suma **3 lub 4** → wiersz 1
- Suma **5 lub 6** → wiersz 2
- Suma **7** → wiersz 3
- Suma **8 lub 9** → wiersz 4
- Suma **10 lub 11** → wiersz 5

Wyjątek: Przy sumie 2 lub 12 gracz **sam wybiera** wiersz klikając w nagłówek.

2.4 Kluczowe zasady

Zasada "typu i kolumny":

Jeśli na kościach wypadnie np. 3 i 5, to:

- Kostka 3 = typ stawu → można postawić staw
- Kostka 5 = kolumna → staw musi być w kolumnie 5

Zasada "alternatywnej kolumny":

Gdy kolumna wynikająca z kości jest pełna, szukamy najbliższej wolnej kolumny (lewo/prawo). Jeśli obie strony mają wolne miejsca:

- **Więcej miejsc po lewej** → tylko lewa dozwolona
- **Więcej miejsc po prawej** → tylko prawa dozwolona
- **Równo** → gracz wybiera (obie podświetlone fioletowo)

Zasada dubla oczek:

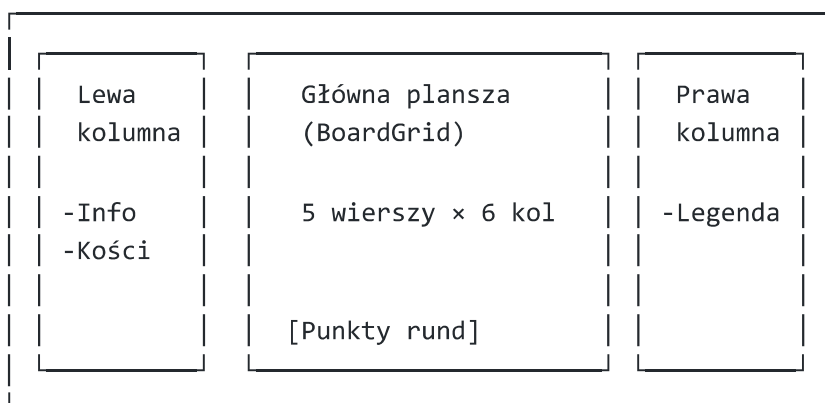
Przy dublu (np. 6,6):

- Projekt podstawowy → kolumna 6 (lub alternatywna)
- Plac → **dowolne puste pole** (bez walidacji kolumny)

3. Stylowanie i UI

3.1 Layout aplikacji

Zastosowaliśmy układ **trzykolumnowy**:



3.2 Komponenty wizualne

BoardGrid - najważniejszy komponent wizualny:

- Siatka CSS Grid (nagłówki + 5 rzędów x 6 kolumn)
- Każda komórka ma:

- o Ikony emoji (🏠 🌲 💧 🧊 🏭)
- o Punkty w prawym górnym rogu
- o Różne kolory tła dla typów
- o Hover effect (powiększenie 1.05x)

Podświetlenia:

- **Żółte** - aktywny wiersz (suma kości)
- **Fioletowe** - alternatywne kolumny
- **Niebieskie** - wybór wiersza przy sumie 2/12
- **Pomarańczowe** - tymczasowe zmiany (przed zapisem)

3.3 Responsive design

```
@media (max-width: 768px) {  
  .game-container {  
    flex-direction: column; /* Układamy pionowo */  
  }  
  .cell {  
    width: 50px; /* Mniejsze komórki */  
    height: 50px;  
  }  
}
```

4. Kluczowe algorytmy

4.1 Rzut kośćmi

Prosty generator liczb losowych z walidacją:

```
rollDice() {  
  // Blokada - tylko 1 rzut na rundę  
  if (this.diceRolledThisRound) {  
    console.warn('Już rzuciłeś kostkami!')  
    return  
  }  
  
  // Generuj 2 kostki (1-6)  
  this.dice = [  
    Math.floor(Math.random() * 6) + 1,  
    Math.floor(Math.random() * 6) + 1,  
  ]  
  
  this.currentPhase = 'building'
```

```

    this.diceRolledThisRound = true
    this.selectedRowIndex = null // Reset wyboru wiersza
  }

```

Dlaczego tak?

Najpierw sprawdzamy, czy nie było już rzutu (`diceRolledThisRound`). Potem generujemy dwie liczby 1-6 i przechodzimy w fazę budowania. Reset `selectedRowIndex` jest ważny, bo przy nowym rzucie gracz musi ponownie wybrać wiersz jeśli trafi 2 lub 12.

4.2 Walidacja kolumn - logika podstawowa

To jest serce gry. Algorytm określa, które kolumny są dostępne dla danego projektu:

```

allowedColumns: (state: GameState) => {
  const info = {
    primary: [] as number[], // Głównie kolumny
    alt: [] as number[],     // Alternatywne
    anyForSquare: false      // Plac wszędzie?
  }

  const [die1, die2] = state.dice

  // Określ typ projektu na podstawie kości
  if (state.selectedProject === 'house') {
    // Dom = 1 lub 4
    // Jeśli die1=1, to kolumna=die2
    // Jeśli die2=1, to kolumna=die1
    if (die1 === 1 || die1 === 4) info.primary.push(die2)
    if (die2 === 1 || die2 === 4) info.primary.push(die1)
  }

  // ... podobnie dla forest (2,5) i lake (3,6)

  // Plac - specjalny przypadek
  if (state.selectedProject === 'square' && die1 === die2) {
    info.anyForSquare = true // Dubel = plac wszędzie
    return info
  }

  return info
}

```

Logika myślenia:

Jedna kostka definiuje **typ** projektu, druga **kolumnę**. Przy kościach 3,5 :

- Jeśli wybieram staw (3) → kolumna 5
- Jeśli wybieram las (5) → kolumna 3

Przy dublu oczek (np. 4,4) plac może być **wszędzie** (`anyForSquare = true`).

4.3 Algorytm alternatywnych kolumn

Gdy kolumna podstawowa jest pełna, szukamy najbliższej wolnej:

```
// Znajdź kolumny alternatywne
info.primary.forEach((col) => {
  if (!colHasSpace(col)) {
    // Szukaj w promieniu 1, 2, 3...
    for (let r = 1; r < 6; r++) {
      const left = ((col - 1 - r + 6) % 6) + 1
      const right = ((col - 1 + r) % 6) + 1

      const leftFree = colHasSpace(left)
      const rightFree = colHasSpace(right)

      // Reguła a) - jedna pełna, druga wolna
      if (leftFree && !rightFree) {
        info.alt.push(left)
        break
      }
      if (!leftFree && rightFree) {
        info.alt.push(right)
        break
      }

      // Reguły b) i c) - obie wolne
      if (leftFree && rightFree) {
        const leftCount = countFreeSpaces(left)
        const rightCount = countFreeSpaces(right)

        if (leftCount > rightCount) {
          info.alt.push(left) // Tylko lewa
        } else if (rightCount > leftCount) {
          info.alt.push(right) // Tylko prawa
        } else {
          info.alt.push(left, right) // Obie
        }
        break
      }
    }
  }
})
```

Jak to działa?

Zawijamy planszę w okrąg (% 6), więc kolumna 1 sąsiaduje z 6. Sprawdzamy po kolei odległość 1, 2, 3... aż znajdziemy wolną. Jeśli obie strony mają wolne miejsca, liczymy je i wybieramy tę z większą liczbą (lub obie, jeśli równo).

4.4 Grupowanie i punktacja BFS

To najbardziej złożony algorytm - liczy punkty za połączone projekty:

```
calculateRoundScore() {  
    const grouped = new Set<string>() // Zapamiętuję co już policzyłem  
  
    // BFS - przeszukiwanie wszerz  
    const groupProjects = (startRow: number, startCol: number, type: CellType) => {  
        const queue = [[startRow, startCol]]  
        const visited = new Set<string>()  
        let groupPoints = 0  
  
        while (queue.length > 0) {  
            const [row, col] = queue.shift()!  
            const cell = board[row][col]  
  
            groupPoints += cell.points // Dodaj punkty z tego pola  
  
            // Sprawdź 4 sąsiadów (N, S, E, W)  
            const neighbors = [  
                [row - 1, col], // Góra  
                [row + 1, col], // Dół  
                [row, col - 1], // Lewo  
                [row, col + 1], // Prawo  
            ]  
  
            for (const [nRow, nCol] of neighbors) {  
                const key = `${nRow},${nCol}`  
                if (visited.has(key)) continue  
  
                const neighbor = board[nRow]?.[nCol]  
                if (neighbor?.type === type && neighbor.occupied) {  
                    visited.add(key)  
                    grouped.add(key)  
                    queue.push([nRow, nCol])  
                }  
            }  
        }  
  
        return groupPoints  
    }  
  
    // Dla każdego pola w punktowanym wierszu  
    for (let col = 0; col < 6; col++) {  
        const cell = board[scoredRow][col]  
        if (cell.occupied && !grouped.has(`${scoredRow},${col}`)) {  
            const score = groupProjects(scoredRow, col, cell.type)  
            roundScore += score  
        }  
    }  
}
```


BFS to klasyka:

Zamiast rekurencji używam kolejki. Zaczynam od jednego pola, dodaję punkty, sprawdzam sąsiadów. Jeśli sąsiad jest tego samego typu, dodaję go do kolejki. `visited` zapobiega zapętleniu, `grouped` zapobiega podwójnemu liczeniu grup.

4.5 Bonusy końcowe - place i fabryki

Po 9 rundach liczę bonusy za place i fabryki:

```
calculateFinalScore() {  
  // Funkcja sprawdzająca sąsiadów N,S,E,W  
  const getNeighbors = (row, col) => {  
    const types = []  
    for (const [r, c] of [[row-1,col], [row+1,col], [row,col-1], [row,col+1]]) {  
      const cell = board[r]?.[c]  
      if (cell?.occupied) types.push(cell.type)  
    }  
    return types  
  }  
  
  // Plac: +10 jeśli sąsiaduje z domem, lasem I stawem  
  if (cell.type === 'square') {  
    const n = getNeighbors(row, col)  
    if (n.includes('house') && n.includes('forest') && n.includes('lake')) {  
      squareBonus += 10  
    }  
  }  
  
  // Fabryka: +10 za las/staw, kary za dom/plac  
  if (cell.type === 'factory') {  
    const n = getNeighbors(row, col)  
  
    if (n.includes('house') || n.includes('square')) {  
      const houseCount = n.filter(x => x === 'house').length  
      const squareCount = n.filter(x => x === 'square').length  
      factoryBonus -= (houseCount * 2 + squareCount * 5)  
    } else if (n.includes('forest') || n.includes('lake')) {  
      factoryBonus += 10  
    }  
  }  
}
```

Strategia:

Place są trudne - muszą mieć wszystkie 3 typy w sąsiedztwie. Fabryki to balans ryzyka: +10 za las/staw, ale -2 za każdy dom i -5 za każdy plac w okolicy.

5. Architektura i state management

5.1 Pinia Store

Cała logika gry jest w jednym store - `GameStore.ts` (1393 linii). Wybraliśmy Pinię zamiast Vuex, bo:

- Lepsza integracja z TypeScript
- Prostsze API (bez mutations)
- Automatyczne devtools

Główne sekcje store:

```
export const useGameStore = defineStore('game', {
  state: () => ({
    currentRound: 0,
    dice: [null, null],
    currentPhase: 'planning' | 'building' | 'bonus' | 'scoring',
    players: [],
    selectedProject: null,
    tempChanges: [],
    // ... itd
  }),

  actions: {
    rollDice() { /* ... */ },
    placeProjectTemp() { /* ... */ },
    saveChanges() { /* ... */ },
    calculateRoundScore() { /* ... */ },
    // ... 20+ akcji
  },

  getters: {
    diceSum: (state) => state.dice[0] + state.dice[1],
    canRollDice: (state) => !state.diceRolledThisRound,
    allowedColumns: (state) => { /* skomplikowana logika */ },
    // ... 10+ getterów
  }
})
```

5.2 Przepływ danych

```
User Action (kliknięcie)
  ↓
Component emit event
  ↓
GameBoard.vue handler
  ↓
Store action
  ↓
State mutation
  ↓
```

Getters **recalculation**
↓
Computed **properties update**
↓
DOM **re-render**

Vue automatycznie śledzi zależności i przelicza tylko to, co się zmieniło.

5.3 Najważniejsze decyzje projektowe

1. Single source of truth

Cały stan gry w jednym miejscu (`GameStore`). Komponenty tylko wyświetlają i emitują eventy.

2. Fazy gry

Rozróżnienie `planning`, `building`, `bonus`, `scoring` upraszcza logikę - każda faza ma inne zasady.

3. Tymczasowe zmiany

`tempChanges[]` przechowuje ruchy przed zapisem. Gracz może je anulować (`clearChanges`).

4. Walidacja w 2 miejscach

- **Gettery** określają, które kolumny są dozwolone
- **Actions** sprawdzają, czy można postawić projekt

To daje natychmiastowy feedback UI (podświetlenia) + finalną walidację przy kliknięciu.

6. Największe wyzwania

6.1 Logika alternatywnych kolumn

To było najtrudniejsze. W zasadach papierowych jest napisane "szukaj najbliższej", ale co jeśli obie strony mają wolne miejsca? Musieliśmy dodać reguły:

- Jeśli jedna strona ma więcej miejsc → tylko ta
- Jeśli równo → gracz wybiera (obie podświetlone)

Implementacja z zawijaniem planszy (`% 6`) i liczeniem wolnych miejsc zajęła kilka iteracji.

6.2 Obsługa dubli

Gra ma 3 typy dubli:

1. **Dubel oczek** (3,3 / 4,4 / 5,5 / 6,6) → plac gdziekolwiek
2. **Dubel projektowy** (1,4 / 2,5 / 3,6) → fabryka w obu kolumnach
3. **Sumy 2 i 12** → gracz wybiera wiersz

Każdy wymaga innej logiki. Debugowanie zajęło sporo czasu, bo łatwo coś przeoczyć.

6.3 BFS dla grupowania

Początkowo próbowaliśmy rekurencji, ale Vue ma problemy z głęboką rekurencją (stack overflow przy dużych grupach). BFS z kolejką jest bezpieczniejsze i łatwiejsze do debugowania.

6.4 TypeScript strict mode

Wszystkie typy muszą być zdefiniowane - czasem frustrujące, ale łapie błędy przed uruchomieniem. Warto było.

7. Przykłady

7.1 Plansza w trakcie gry

Nagłówki kolumn:	1	2	3	4	5	6
Wiersz 3,4 (3pkt)						
Wiersz 5,6 (1pkt)						
Wiersz 7 (3pkt)						
Wiersz 8,9 (1pkt)						
Wiersz 10,11 (3p)						

← Aktywny

Wiersz 7 jest żółty (podświetlony), bo wypadła suma 7 na kościach.

7.2 Przykład punktacji

Runda 5: suma kości = 7 (wiersz 3)

Wiersz 3:  | _ |  |  | _ | 

Grupowanie:

- Staw lewy (1 pole, 2 pkt) → 2 pkt
- Dom środek (2 pola, 1+1 pkt) → 2 pkt (grupa!)
- Staw prawy (1 pole, 2 pkt) → 2 pkt

Suma: 6 punktów

Gdyby domy były oddzielone, każdy by dał 1 pkt (razem 2). Ale są razem, więc **grupowanie się** opłaca.

8. Podsumowanie

8.1 Co się udało

- **Pełna funkcjonalność** - wszystkie zasady gry zaimplementowane
- **Czytelny kod** - TypeScript + dobre nazewnictwo
- **Responsywność** - działa na telefonie
- **Płynna rozgrywka** - brak bugów, intuicyjny UX

8.2 Co można by poprawić

- **Internationalization** - aktualnie UI po polsku, można dodać i18n
- **Animacje** - przejścia między fazami mogłyby być płynniejsze
- **Historia ruchów** - możliwość cofnięcia się
- **Multiplayer** - WebSockets + współdzielona plansza

8.3 Czego się nauczyliśmy

1. **Vue 3 Composition API** - bardziej elastyczne niż Options API
2. **Pinia** - prostsza od Vuex, lepsza dla TypeScript
3. **Algorytmy grafowe** - BFS w praktyce
4. **State management** - jak zarządzać złożonym stanem aplikacji
5. **TypeScript strict** - wymaga więcej pracy, ale łapie błędy wcześniej

8.4 Świadome wybory

- **Jedna plansza zamiast wielu graczy** - uproszczenie dla MVP
- **Emoji zamiast grafik** - szybszy development, działa wszędzie
- **CSS Grid zamiast Canvas** - łatwiejsze stylowanie i responsywność
- **Store-centric architecture** - łatwiejsze testowanie i debugging

9. Kod źródłowy i dokumentacja

Repozytorium: <https://github.com/TurboSebo/my-little-town>

Kluczowe pliki:

- `src/stores/GameStore.ts` - cała logika (1393 linii)
- `src/components/BoardGrid.vue` - główna plansza
- `README.md` - pełna dokumentacja techniczna (579 linii)

Statystyki:

- ~3000 linii kodu (TypeScript + Vue)
- ~800 linii CSS
- 9 komponentów Vue
- 25+ akcji w store
- 12 getterów

10. Uruchomienie projektu krok po kroku

```
# 1. Wymagania
node --version # Powinna być 20+ lub 22+

# 2. Clone
git clone https://github.com/TurboSebo/my-little-town.git
cd my-little-town

# 3. Install
npm install

# 4. Dev server
npm run dev

# 5. Otwórz w przeglądarce
# http://localhost:5173

# 6. Production build
npm run build
npm run preview # Podgląd buildu
```

Typowe problemy:

- **Port zajęty** - Vite automatycznie znajdzie inny
- **Node za stary** - zaktualizuj do 20.19.0+
- **TypeScript errors** - `npm run type-check` pokazuje szczegóły

Podsumowanie autorów

Ten projekt to efekt kilkudziesięciu godzin pracy. Największym wyzwaniem było przeniesienie złożonych zasad gry planszowej do kodu, zachowując przy tym czytelność i możliwość dalszego rozwijania. Udało się stworzyć w pełni funkcjonalną grę, która odwzorowuje mechanikę oryginału.

Autorzy: Marek Otulak, Sebastian Kałuża

PSAM 2025